

RAGAS로 RAG 성능 평가하기

RAG 시스템의 품질을 객관적으로 측정하는 방법

RAGAS

Ground Truth

팀별 성능 평가

목차

01

RAGAS란?

기본 개념 & 배경

02

핵심 평가 지표

4가지 메트릭 이해

03

RAGAS 기본 코드

설치 & 실습 코드

04

팀별 성능 평가

Ground Truth & 실전 방법

RAGAS = "RAG를 평가(Assess)하는 Python 프레임워크"

RAG 파이프라인의 각 단계를 LLM이 자동으로 채점 | 0~1 사이 점수로 출력

?

문제

RAG 성능이 좋은지
나쁜지 어떻게 알까?

👁👁

기존 방법

사람이 직접 읽고 평가
→ 느리고 주관적

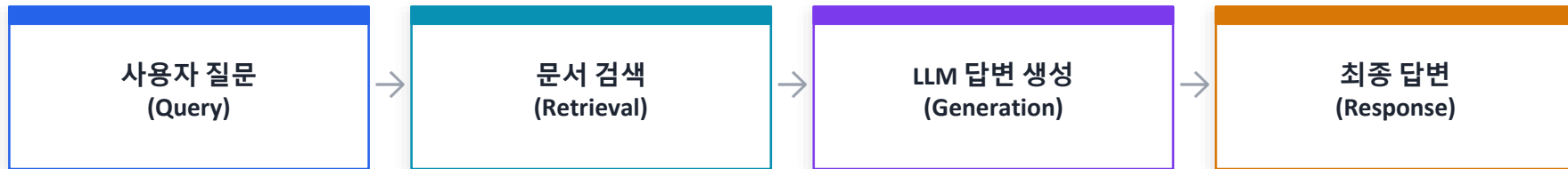
📦?

RAGAS 해결책

LLM이 자동으로
객관적 점수 생성!

RAGAS가 평가하는 RAG 파이프라인

어느 단계에서 문제가 생겼는지 파악 가능



RAGAS 평가 포인트

검색 품질 평가

Context Precision
Context Recall

답변 품질 평가

Faithfulness
Answer Relevancy

종합 점수

0 ~ 1 사이 점수
높을수록 우수

핵심 평가 지표 4가지

각 지표는 0~1 사이 점수 | 1에 가까울수록 우수

Faithfulness

충실도

생성된 답변이 검색된 문맥에 근거하고 있는가?

답변이 맥락을 벗어나 '환각(hallucination)' 없이 생성됐는지 측정

Answer Relevancy

답변 관련성

답변이 질문에 대해 얼마나 관련성이 있는가?

핵심을 잘 짚었는지, 불필요한 내용은 없는지 측정

Context Precision

문맥 정밀도

검색된 문맥 중 관련 있는 내용이 앞에 있는가?

관련 청크가 상위에 랭크되어 있는지 측정

Context Recall

문맥 재현율

정답에 필요한 정보가 검색된 문맥에 포함됐는가?

Ground Truth 생성에 필요한 정보가 모두 검색됐는지 측정

1

설치

2

API 키 설정

3

Import

4

데이터 준비

```
# 1. 설치
pip install ragas langchain openai datasets

# 2. OpenAI API 키 설정
import os
os.environ["OPENAI_API_KEY"] = "sk-..."

# 3. RAGAS import
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall
)

# 4. 데이터셋 구조 (4개 필드 필수)
data = {
    "question": 질문 리스트,
    "answer": LLM 답변 리스트,
    "contexts": [[검색된 청크들]],
    "ground_truth": 정답 리스트
}
```

RAGAS 평가 실행 전체 코드

복사해서 바로 실행 가능한 최소 예제

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

# 1. 평가 데이터 준비 (예시 1개)
data = {
    "question": ["파이썬에서 리스트와 튜플의 차이는?"],
    "answer": ["리스트는 변경 가능, 튜플은 변경 불가"],
    "contexts": [["리스트(list)는 mutable... 튜플(tuple)은 immutable..."]],
    "ground_truth": ["리스트는 mutable이고 튜플은 immutable이다"]
}
dataset = Dataset.from_dict(data)

# 2. RAGAS 평가 실행
result = evaluate(dataset, metrics=[faithfulness, answer_relevancy, context_precision, context_recall])

# 3. 결과 출력
print(result) # {'faithfulness': 0.92, 'answer_relevancy': 0.88, ...}
```

Ground Truth란?

팀별로 직접 만들어야 하는 '정답 데이터'

Ground Truth = 사람이 직접 작성한 '모범 정답' → RAGAS가 이것을 기준으로 점수를 매김

Ground Truth 없이 평가

- ✓ Faithfulness
- ✓ Answer Relevancy
- ✗ Context Precision
- ✗ Context Recall

→ 검색 품질 평가 불가!

Ground Truth 있는 평가

- ✓ Faithfulness
- ✓ Answer Relevancy
- ✓ Context Precision
- ✓ Context Recall

→ 완전한 4가지 지표 측정 가능!

팀별 Ground Truth 10개 만들기

좋은 테스트셋이 좋은 평가를 만든다!

Step 1

문서 분석

- 팀이 사용하는 문서를 꼼꼼히 읽기
- 핵심 정보가 담긴 단락 10개 선택
- 너무 쉬운/어려운 질문 균형 맞추기

Step 2

질문 & 정답 작성

- 선택한 단락마다 질문 1개 생성
- 그 단락 내용으로 정답 직접 작성
- 명확하고 구체적인 문장으로 서술

Step 3

품질 검토 & 저장

- 다른 팀원이 정답 검토 & 수정
- 정답이 문서 내용에 기반했는지 확인
- 10개 완성 후 CSV/JSON으로 저장

팁: 단순 사실 질문(3개) + 비교/설명 질문(4개) + 추론 질문(3개)로 다양하게 구성

팀별 성능 평가 예시 코드

10개 Ground Truth 기반 전체 평가 파이프라인

```
# 팀별 RAG 성능 평가 전체 파이프라인
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

# 팀이 준비한 10개 Ground Truth + LLM 응답 데이터
team_data = {
    "question":      questions_list,      # 10개 질문
    "answer":        rag_answers_list,     # RAG 시스템 답변 10개
    "contexts":      retrieved_contexts,   # 검색된 문맥 (질문당 여러 청크)
    "ground_truth":  ground_truth_list    # 팀이 작성한 정답 10개
}

dataset = Dataset.from_dict(team_data)
metrics = [faithfulness, answer_relevancy, context_precision, context_recall]
result = evaluate(dataset, metrics=metrics)

print(result.to_pandas()) # 질문별 상세 점수 확인
```

결과 해석 & 팀별 비교

점수를 통해 RAG 파이프라인의 약점을 찾아보자

팀별 결과 예시 (샘플)

팀명	Faithfulness	Answer Rel.	Ctx Precision	Ctx Recall
팀 A	0.92	0.85	0.78	0.81
팀 B	0.75	0.91	0.65	0.70
팀 C	0.88	0.79	0.83	0.86

점수 해석 가이드

- 0.9 이상 → 우수
- 0.7 ~ 0.89 → 양호
- 0.5 ~ 0.69 → 개선 필요
- 0.5 미만 → 요주의

낮은 점수별 개선 방향

Faithfulness 낮음

→ 프롬프트에 '반드시 문맥만 활용' 명시, 환각 방지 지시 추가

Context Precision 낮음

→ 청크 크기 줄이기, 임베딩 모델 교체, Top-K 수 조정

Context Recall 낮음

→ 청크 오버랩 늘리기, 더 많은 청크(K) 검색하도록 변경

Answer Relevancy 낮음

→ 시스템 프롬프트 개선, 답변 형식 & 길이 가이드라인 추가

팀별 결과 시각화 코드

결과를 DataFrame으로 저장하고 막대 그래프로 비교하기

```
import pandas as pd
import matplotlib.pyplot as plt

# 결과를 DataFrame으로 변환
df = result.to_pandas()
print(df[["question", "faithfulness", "answer_relevancy", "context_precision", "context_recall"]])

# 평균 점수 출력
cols = ['faithfulness', 'answer_relevancy', 'context_precision', 'context_recall']
mean_scores = df[cols].mean()
print('팀 평균 점수:')
print(mean_scores.round(3))

# 막대 그래프 시각화
mean_scores.plot(kind='bar', figsize=(8,4),
                  color=['#2563EB', '#0891B2', '#7C3AED', '#D97706'])
plt.title('우리 팀 RAG 성능 평가 결과')
plt.ylim(0, 1.0); plt.xticks(rotation=20)
plt.savefig('team_result.png', dpi=150); plt.show()
```

팀 프로젝트 체크리스트

이것만 지키면 RAGAS 평가 완성!

01 문서 선택

팀이 사용할 문서(PDF, TXT 등) 결정 · LangChain으로 RAG 파이프라인 구축 완료

02 Ground Truth 10개 작성

문서 기반 질문 10개 + 모범 정답 10개 준비 · CSV 또는 JSON 형태로 저장

03 RAGAS 설치 & 환경 설정

pip install ragas datasets 완료 · OpenAI API 키 설정 완료

04 평가 실행 & 결과 저장

evaluate() 함수 실행 후 to_pandas() 저장 · 4가지 지표 점수 기록

05 결과 분석 & 개선

낮은 점수 지표 원인 분석 · 파라미터 개선 후 재평가 비교

좋은 Ground Truth = 좋은 평가 = 좋은 RAG 시스템!